

CS336 作业 5 (对齐): 推理强化学习

版本 26.0.0

CS336 教学团队

2026 年春季

译注: 本译文按原文逐页对应。自然语言提示词已翻译, 模板标签、代码标识符和路径保持原样; 复现实验请使用仓库提供的原始 .prompt 文件。

1 作业概览

在本次作业中, 你将亲手训练语言模型, 使其能够进行推理并解决下游任务。

需要实现的内容

1. 零样本、少样本以及思维链提示。
2. 组相对策略优化 (Group Relative Policy Optimization, GRPO): 一种利用外部奖励来提升模型性能的强化学习算法。
3. 策略梯度估计的不同变体, 用于探索方差缩减与重要性权重裁剪策略。

需要运行的实验

1. 测量 OLMo-2-0425-1B 在 GSM8K 上的提示效果。
2. 在 OLMo-2-0425-1B 上运行同策略 (on-policy) GRPO, 提升其在 GSM8K 上的性能。
3. 运行 RFT、Dr. GRPO 和 MaxRL 等 RL 变体, 探索强化学习中的算法选择。
4. 运行异策略 (off-policy) GRPO, 加速训练并探索不同的裁剪策略。

代码结构

本作业的全部代码与本说明文档均可在 GitHub 获取:

<https://github.com/stanford-cs336/assignment5-alignment>

请使用 `git clone` 克隆仓库。如有更新, 我们会通知你; 届时可使用 `git pull` 获取最新版本。

1. `cs336_alignment/*`: 你将在这里编写作业 5 的代码。注意, 除下述起始代码外, 这里没有其他代码, 因此你可以从零开始, 按自己的方式实现。
2. `cs336_alignment/vllm_utils.py`: 包含运行 vLLM 服务器的代码; 该服务器可生成文本并同步权重。
3. `cs336_alignment/drgrpo_grader.py`: 包含为模型生成的数学题答案评分的代码。
4. `cs336_alignment/prompts/*`: 为方便使用, 我们提供了包含提示词的文本文件。
5. `tests/*.py`: 此目录包含必做的 GRPO 测试, 以及选做的对齐/安全补充作业测试。

`tests/test_grpo.py` 中的必做测试会调用 `tests/adapters.py` 中定义的钩子。你需要实现这些适配器, 将自己的代码连接到测试。编写更多测试和/或修改测试代码有助于调试, 但你的实现应当通过原始提供的测试套件。

1. `README.md`: 包含环境配置的基本说明。

如何提交

你需要向 Gradescope 提交以下文件:

- `writeup.pdf`: 回答所有书面问题。请将答案排版成文档。
- `code.zip`: 包含你编写的全部代码。

运行 `test_and_make_submission.sh` 脚本, 生成 `code.zip` 文件。

2 引言

2.1 背景

在本课程的前四次作业中, 我们学习了如何预训练一个基础模型。现在, 我们已经准备好学习后训练 (post-training): 有了基础模型之后, 怎样把它变成能够解决下游任务的实用工具?

后训练的一个组成部分是对齐 (alignment)。预训练所使用的数据混合与训练目标, 会让模型获得广泛的知识 and 行为模式。然而, 当我们要求模型完成任务时, 希望它表现出的是一种特定行为: 充当有帮助且无害的助手。将通用基础模型转变为聚焦对话的模型, 这一过程称为“指令微调”或“对齐”。我们将在作业 5 的选做补充材料中介绍这些技术。

后训练的另一个组成部分是强化学习。在预训练中, 我们希望赋予模型广泛的知识, 因此使用范围广泛的数据, 例如互联网上的文本。在后训练 RL 中, 目标变得更集中: 我们希望模型在某个具体任务上达到较高准确率, 例如解数学题。这个更集中的目标至少在两个方面不同于预训练: (a) 我们不再拥有同样多的数据; (b) 训练目标从“覆盖” (拥有广泛知识) 转向“精确” (生成准确回答)。由于这些差异, 我们需要一种新技术, 即强化学习 (RL)。

在 RL 中, 我们拥有一个问题数据集, 以及一个判断回答是否正确解决问题的评分函数。例如, 在编程场景中, 我们可能得到一道编程题 (如“编写一个将列表反转的 Python 函数”), 以及由一组测试用例构成的评分函数 (如 `assert f([0, 1, 2]) == [2, 1, 0]`)。在数学场景中, 我们可能得到一道数学推理应用题 (如“四年前汤姆的年龄是约翰的一半, 约翰现在二十岁, 那么汤姆现在几岁?”), 以及一个解析最终答案并检查其是否正确 (12) 的评分函数。值得注意的是, 与预训练不同, 我们没有一份可供模仿的回答数据集: 在编程示例中, 没有给出能够解决任务的正确 Python 程序; 在数学推理示例中, 也没有给出产生答案的正确推理链。相反, 在 RL 中, 我们直接以模型准确率为目标函数进行梯度更新。从高层次看, RL 会从模型中采样回答, 使用评分函数打分, 然后提高正确回答的权重。

本次作业将从数学和实验两个方面介绍 RL。事实证明, RL 很困难: 它既缓慢, 又不稳定。开展 RL 科学研究也很困难, 因为不同随机种子的运行结果方差较大, 而且实现中看似微小的细节也会产生重大影响。本作业将介绍大语言模型强化学习, 并探索其中的一些挑战。

2.2 模型与数据集

本次作业将使用 OLMo-2-0425-1B。这个基础模型先在 OLMo-mix-1124 上进行预训练, 再在 Dolmino-mix-1124 上进行中期训练, 总计训练了 4 万亿个 token。OLMo-mix-1124 主要由课堂上介绍过的 DCLM-Baseline 数据集组成 [J. Li 等, 2024]; Dolmino-mix-1124 则是一种更聚焦的数据混合, 大约 50% 为 DCLM, 另外 50% 为指令遵循、数学、代码、STEM 论文及维基数据。OLMo-2-0425-1B 的其他细节可参见 OLMo 2

技术报告 [T. OLMo 等, 2024]。学到课程的这个阶段，你应该已经具备理解其中每项设计决策所需的背景知识。

下游任务将使用 GSM8K 数据集 [K. Cobbe 等, 2021]。你可以在作业仓库的 `data/gsm8k/train.jsonl` 和 `data/gsm8k/test.jsonl` 中找到它，也可以在线获取。该数据集由相对简单的小学数学推理应用题组成。下面是一个示例：

```
{
  "question": "纳塔莉娅在四月向她的 48 位朋友卖出了发夹，五月卖出的发夹数量是四月的一半。她在四月和五月总共卖出了多少个发夹？",
  "answer": "纳塔莉娅在五月卖出了 48/2 = <<48/2=24>>24 个发夹。\\n她在四月和五月总共卖出了 48+24 = <<48+24=72>>72 个发夹。\\n#### 72"
}
```

在 RL 过程中，模型将学习为这类问题生成推理链，从而提高解决数学问题的能力。

关于模型与数据集选择的说明：受资源限制，我们只选择了一个小规模“模型—数据集”组合，作为 RL 的试验平台。具体而言，我们选择的模型虽然规模小，但训练 token 数非常多（达到参数量的 4000 倍！），因而具有较强能力，使我们能在小规模条件下、在真实数据集上观察到合理的 RL 效果。遗憾的是，我们在本课程中训练的模型还不足以解决数学问题。完成本次作业后，有兴趣的同学也可以拿自己在课程中训练的模型，在更简单的任务上运行 RL。由于 RL 的训练动态高度依赖模型和数据集，请记住，本作业中观察到的结果未必能够迁移到其他模型—数据集组合。

2.3 记号

本作业将涉及一些数学内容，因此下表列出了后续要使用的部分记号。语言建模与强化学习往往对同一对象使用不同术语，表中同时列出了两种说法。忘记某个记号的含义时，可以随时回来查阅。

记号	语言模型术语	强化学习术语	含义
ρ	提示分布或数据集	初始状态分布	提示／问题的分布。
x	提示；问题	初始状态	从 ρ 中采样的提示／问题。
y	回答；补全；生成结果；样本	rollout；轨迹；采样得到的动作序列	针对提示 x 采样得到的回答。
y_t	token	动作	y 中第 t 个生成的 token。
$y_{<t}$	前缀		位置 t 之前所有已生成的 token，即 $y_1, \dots, y_{t-1}$ 。
π_θ	模型	策略	参数为 θ 的模型；给定提示 x ，为回答 y 分配概率 $\pi_\theta(y \mid x)$ 。

$\pi_{\theta}(y_t \mid x, y_{<t})$	下一个 token 的分布	时刻 t 的策略	给定提示及之前生成的 token 时, token y_t 的条件概率。
$r(y \mid x)$		奖励	表示采样回答是否正确的标量分数 (本作业中为 0 或 1)。
B	每批提示数		每个推理批次中的提示数量。
G	每个提示的生成数	组大小	对每个提示采样得到的回答数量。
$\text{len}(y)$ 或 L	回答长度	时域长度 (horizon)	一个回答中生成的 token 数量。
$A^{(i,j)}$		优势	经过基线处理与归一化后, 分配给提示 i 的第 j 个回答的权重。

3 提示

将预训练基础模型用于下游任务的第一步, 是向它提供提示。基础模型在预训练时学到了广泛的行为, 提示是一种轻量级手段, 可将模型行为引导到解决任务的模式。后续作业中, 我们也会看到, 提示的选择会影响 RL 的训练动态与探索过程。

最基本的提示方法是提供问题, 然后从模型的下一个 token 分布中采样, 生成答案; 我们将这种策略称为 `question_only`。我们会把它与 `r1_zero` 提示进行比较, 后者同时包含问题和要求模型进行思维链推理的指令 [DeepSeek-AI 等, 2025]。

3.1 使用 vLLM 进行推理

要让模型生成回答, 我们需要一个推理引擎。实现推理引擎超出了本次作业的范围, 因此我们将使用 vLLM 推理引擎 [W. Kwon 等, 2023]。它实现了多种优化, 包括快速 CUDA 核函数、用于高效注意力 KV 缓存的 PagedAttention 等。启动 vLLM 服务器并生成文本的代码已经提供在 `cs336_alignment/vllm_utils.py` 中, 接口如下:

```
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None

@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9
```

```
def start(self) -> None: ...
def generate_completions(
    self,
    prompts: list[str],
    sampling_params: dict,
    batch_size: int | None = None,
) -> list[VLLMCompletion]: ...
```

3.2 零样本、少样本与思维链提示

除非另有说明, GSM8K 实验将使用下面来自 DeepSeek R1-Zero 模型的提示 [DeepSeek-AI 等, 2025]。我们将其称为 `r1_zero` 提示:

```
User 与 Assistant 之间的一段对话。User 提出问题, Assistant 解决问题。Assistant 首先在心中思考推理过程, 然后向 User 提供答案。
推理过程和答案分别放在 <think> </think> 与 <answer> </answer> 标签中, 即 <think> 这里是推理过程 </think> <answer> 这里是答案
</answer>。
User: {question}
Assistant: <think>
```

`r1_zero` 提示位于文本文件 `cs336_alignment/prompts/r1_zero.prompt` 中。

提示中的 `question` 表示我们插入的某个问题 (例如: “纳塔莉娅在四月向她的 48 位朋友卖出了发夹, 五月卖出的发夹数量是四月的一半。她在四月和五月总共卖出了多少个发夹?”)。我们期望模型扮演助手, 开始生成思考过程 (因为我们已经加入了开头的 `<think>` 标签), 用 `</think>` 结束思考, 再在 `answer` 标签内生成最终的符号形式答案, 例如 `<answer> 4x + 10 </answer>`。要求模型生成 `<answer> </answer>` 等标签, 是为了便于解析输出并与标准答案比较, 同时也便于在遇到结束标签 `</answer>` 时停止生成。

另一种提示方式称为少样本提示 (few-shot prompting): 在向模型提出实际问题之前, 先放入几个问题—答案对。`r1_zero` 提示的少样本版本如下:

```
User 与 Assistant 之间的一段对话。User 提出问题, Assistant 解决问题。Assistant 首先在心中思考推理过程, 然后向 User 提供答案。
推理过程和答案分别放在 <think> </think> 与 <answer> </answer> 标签中, 即 <think> 这里是推理过程 </think> <answer> 这里是答案
</answer>。
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

少样本提示通过提供几个待解决任务的示例，改善模型表现。开放基础模型所报告的基准结果往往采用少样本设置：例如，OLMo-2-0425-1B 模型卡报告的 GSM8K 结果采用 8-shot 提示。我们在 `cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt` 中提供了 `r1_zero` 的 3-shot 版本，示例取自 OLMES 仓库。

最后，作为基线，我们还将使用 `question_only` 提示，其文件位于 `cs336_alignment/prompts/question_only.prompt`：

```
{question} 请将你的最终答案放在 \boxed{} 中。
```

注意，虽然该提示名为 `question_only`，我们仍然要求将最终答案放入方框，以便评分器从回答中解析答案。下一节将对此进行说明。

3.3 评分函数

模型生成回答后，我们需要检查回答是否正确。数学题带有标准答案（例如 0.5），但模型可以用多种形式给出正确答案，例如 `<answer> 1/2 </answer>` 或“答案是 0.5”。因此，为了正确评分，我们需要一个答案解析函数：它接收模型输出和已知标准答案，返回表示模型输出是否正确的布尔值。

实验将使用近期推理 RL 工作中的一个快速且相当准确的答案解析器 [Z. Liu 等, 2025]。对于 `r1_zero` 提示，该奖励函数实现于 `cs336_alignment.drgrpо_grader.r1_zero_reward_fn`。`question_only` 提示不要求使用 `<think>` 和 `<answer>` 标签，因此应使用同一评分文件中的 `cs336_alignment.drgrpо_grader.question_only_reward_fn` 进行评估。注意，`question_only` 解析器会在 `\boxed{}` 中寻找最终答案。这些奖励函数会返回总奖励，以及独立的格式奖励与答案奖励，分别表示回答是否符合预期格式、解析出的答案是否正确。一些研究会对格式正确但答案错误的回答给部分分数（非零总奖励）。本实验不提供部分分数，因此总奖励就是答案奖励，格式奖励仅用于日志记录。

注意，这些奖励函数要求 `ground_truth` 参数仅包含答案。GSM8K 数据集中的标准回答形式为 `{rationale} #### {answer}`；提取答案时，应以 `####` 分割，并去除首尾空白。

3.4 实验

现在，我们已经可以评估基础模型在不同提示下的表现。

生成超参数。生成回答时使用温度 1.0、top-p 1.0、最大生成长度 512。`r1_zero` 提示要求模型以 `</answer>` 结束答案，因此使用这些提示时，可以让 vLLM 在输出该字符串后停止：

```
# 参考 Dr. GRPO: 模型完成答案时停止
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

只对 `r1_zero` 和 `r1_zero_three_shot` 使用这个停止字符串，不要对 `question_only` 使用。

题目 (prompting_baselines): 在 GSM8K 上运行 OLMo-2-0425-1B (5 分)

(a) 编写脚本, 评估 OLMo-2-0425-1B 在 GSM8K 上使用零样本 question_only、零样本 r1_zero 和少样本 r1_zero_three_shot 提示时的性能。

然后运行脚本并观察输出。对于每种提示, 分别有多少个生成结果属于以下类别: (1) 格式奖励与正确性奖励均为 1; (2) 格式奖励为 1、正确性奖励为 0; (3) 格式奖励与正确性奖励均为 0? 至少观察类别 2 中的十个示例, 其中多少个输出实际上正确, 只是未被正确解析? 类别 3 的情况又如何?

提交内容: 用几句话说明, 给出评估指标, 以及若干提示和回答示例。

(b) 观察模型输出, 描述每种提示下模型的行为。例如, 我们希望模型回答问题时, 仅提供问题是否足够? 模型是否会表现出回答问题以外的行为? 零样本 r1_zero 和少样本 r1_zero_three_shot 提示如何塑造模型行为?

提交内容: 用几句话评论, 并提供支持结论的示例。

4 组相对策略优化

在测量仅使用提示时模型的表现之后, 下一步是通过训练来提升性能。具体来说, 我们希望优化模型准确率, 等价地说, 就是优化期望奖励:

$$J_{\theta} = E_{x \sim \rho} E_{y \sim \pi_{\theta}(y | x)} [r(y | x)], \quad (1)$$

其中, ρ 是定义在提示/问题 x 上的任务分布, π_{θ} 是模型, y 是针对 x 采样得到的回答/解答, $r(y | x)$ 表示 y 是否正确回答了 x (我们也将 r 称为“奖励函数”)。注意, 该目标与本课程此前研究的预训练目标大不相同: 交叉熵损失 $E_{x \sim \mathcal{D}} [-\log \pi_{\theta}(x)]$ 从数据集 $\mathcal{D}$ 中抽取样本, 而准确率目标对模型自身产生的样本取期望。因此, 我们需要新的优化工具, 具体而言就是强化学习 (RL)。从高层次看, RL 会从模型中采样回答, 用评分函数打分, 再强化正确回答。RL 往往很慢且具有挑战性, 因此本作业的大量内容都围绕如何使其更快、更稳定展开。

RL 的另一个特点是: 由于它需要从模型中采样, 采样时使用的提示会影响训练动态。语言模型研究中一个令人振奋的发现是, 使用思维链推理提示并在强大的基础模型上执行 RL, 可以显著提高模型的推理能力 [OpenAI 等, 2024; DeepSeek-AI 等, 2025]。因此, 除了探索 RL 的核心算法思想, 我们还会探索它与思维链提示等语言建模技术之间的相互作用。

4.1 推导同策略 GRPO

我们将首先探索 RL 背后的数学, 逐步构建组相对策略优化 (GRPO) 算法。这是在训练语言模型时常用的一种 RL 算法 [Z. Shao 等, 2024]。

我们的讲解紧密参考了两份出色的资料，它们更深入地介绍了这些概念：OpenAI 的《Spinning Up in Deep RL》[J. Achiam, 2018]，以及 Nathan Lambert 的《Reinforcement Learning from Human Feedback (RLHF)》一书 [N. Lambert, 2024]。

4.1.1 将语言模型视为策略

在传统强化学习中，我们考虑的是一个接收状态 s_t 并输出动作 a_t 的策略。该动作随后产生奖励 $r_t = r(s_t, a_t)$ ，并从某个下一状态分布中产生后续状态 $s_{t+1} \sim \text{next_state}(s_t, a_t)$ 。强化学习就是优化策略，使其获得较高奖励。

参数为 θ 的因果语言模型 (LM) π_θ ，定义了给定当前文本前缀 $y_{<t} = (y_1, \dots, y_{t-1})$ 时下一个 token y_t 的概率分布。因此，在 RL 语境下，可以将下一个 token y_t 视为动作 a_t ，将当前文本前缀 $y_{<t}$ 视为状态 s_t 。于是，语言模型就是一个类别分布上的随机策略：

$$a_t \sim \pi_\theta(\cdot | s_t), \pi_\theta(a_t | s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}. \quad (2)$$

使用策略梯度优化策略时，需要两项基本操作：

1. **从策略中采样**：从上述类别分布中抽取动作 a_t ；
2. **计算动作的对数似然**：求值 $\log \pi_\theta(a_t | s_t)$ 。

一般来说，在对 LLM 进行 RL 时， s_t 是截至当前生成的部分补全/解答，每个 a_t 是解答的下一个 token；当模型输出文本结束 token 时，一个回合结束，例如 `<|end_of_text|>`，或在 `r1_zero` 提示中输出 `</answer>`。

4.1.2 轨迹

在强化学习中，我们从某个起始分布抽取初始状态 $s_0 \sim \rho$ 。随后，从策略 π_θ 中采样动作 a_t ，按照下一状态分布 $s_{t+1} \sim \text{next_state}(s_t, a_t)$ 转移到下一个状态，并重复这一过程。这一系列状态与动作构成一条（有限时域）轨迹：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T), \quad (3)$$

其中， T 为轨迹长度，即 a_T 是文本结束 token，或者此时已经达到以 token 数计的最大生成预算。

在我们的场景中， $s_0 \sim \rho$ 就是包含待解数学题的提示 x （可能应用了 CoT 提示或少样本提示）。从这个前缀开始，我们逐个采样 token；下一个状态就是旧前缀拼接新输出的 token，即 $s_{t+1} = (s_t, a_t)$ 。轨迹也称为回合 (episode) 或 rollout；这些术语在本文中交替使用。

4.1.3 奖励与回报

在 RL 中，标量奖励 $r_t = r(s_t, a_t)$ 用于评价在状态 s_t 执行动作的即时质量。对于可验证领域中的 RL，例如本作业的数学应用题任务，中间推理步骤的奖励均为零，直到输出答案。此时，终止动作会获得一个可验证的奖励：

$$r_T = r(s_T, a_T) := \begin{cases} 1 & \text{若根据奖励函数，轨迹 } \tau \text{ 与标准答案匹配;} \\ 0 & \text{否则。} \end{cases} \quad (4)$$

在我们的场景中，最终答案正确时 $r_T = 1$ ，否则为 0。回报 $R(\tau)$ 聚合轨迹上的奖励，在这里就等于 r_T 。

智能体的目标是最大化期望回报：

$$J_\theta = E_{\tau \sim \pi_\theta}[r(\tau)], \quad (5)$$

其中， $\tau \sim \pi_\theta$ 表示如下过程定义的轨迹分布：先采样 $s_0 \sim \rho$ ，再从策略中采样动作，并从下一状态分布中采样后续状态。在我们的场景中，这意味着先从数据集采样数学题 $x \sim \rho$ ，再持续采样 token，直到回答结束；这一过程可以记为 $y \sim \pi_\theta(y | x)$ 。该目标对应以下优化问题：

$$\theta^* = \arg \max_{\theta} J_\theta. \quad (6)$$

4.1.4 策略梯度

现在，我们已经拥有学习“策略梯度”这一 RL 算法所需的记号与定义。在作业剩余部分，我们将坚持使用语言模型记号 (x, y) 来表示提示与回答，而不是使用 RL 中的状态 s_t 与动作 a_t 记号。

回顾一下，我们希望优化模型的期望奖励（即准确率）：

$$J_\theta = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x)], \quad (7)$$

其中， $r(y | x)$ 表示回答 y 对提示 x 是否正确。我们的方法是对目标做梯度上升，即：

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J_{\theta_k}, \quad (8)$$

但要做到这一点，必须能够根据样本估计梯度 $\nabla_{\theta} J_{\theta_k}$ 。将该梯度改写为对样本的期望，就会得到一个称为“REINFORCE 策略梯度”的表达式，名称来自 REINFORCE 论文 [R. J. Williams, 1992]。具体地，梯度可以改写为：

$$\nabla_{\theta} E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)]. \quad (9)$$

在推导之前，先注意这个表达式给出了一种直观算法（REINFORCE）：从数据集采样问题 x ，从模型采样回答 y ，在样本上计算 $r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)$ 的平均值，然后执行一次梯度更新。表达式 $r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)$ 表明，我们应提高高奖励 $r(y | x)$ 的回答 y 的对数概率。重复这一流程并执行多次梯度更新，就得到了基本的强化学习训练循环。

推导 REINFORCE 策略梯度时，可以使用对数导数技巧（回顾： $\log f(x)$ 的导数为 $f'(x)/f(x)$ ）：

$$\nabla_{\theta} \pi_\theta(y | x) = \pi_\theta(y | x) \nabla_{\theta} \log \pi_\theta(y | x). \quad (10)$$

直接应用这个恒等式，即可得到所需结果：

$$\nabla_{\theta} E_{y \sim \pi_\theta(y | x)}[r(y | x)] = \sum_y \nabla_{\theta} \pi_\theta(y | x) r(y | x) \quad (11)$$

$$= \sum_y \pi_\theta(y | x) \nabla_{\theta} \log \pi_\theta(y | x) r(y | x) \quad (12)$$

$$= E_{y \sim \pi_\theta(y | x)}[r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)]. \quad (13)$$

注意，在这些等式中，我们省略了对提示的期望 $E_{x \sim \rho}$ ，因为 ∇_{θ} 与 $E_{x \sim \rho}$ 可以交换顺序。

因此，给定一批提示 $x^{(1)}, \dots, x^{(B)} \stackrel{\text{iid}}{\sim} \rho$ ，并为每个提示采样 G 个回答 $y^{(i,j)} \stackrel{\text{iid}}{\sim} \pi_\theta(y | x^{(i)})$ ，可得到以下策略梯度的样本估计：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G r(y^{(i,j)} | x^{(i)}) \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)}), \quad (14)$$

它会提高高奖励回答的权重。为了推导 GRPO，我们将以这个基本策略梯度估计器为起点，逐项进行修改。

4.1.5 基线

基本 REINFORCE 策略梯度估计器的期望虽然正确，等于 $\nabla_\theta J_\theta$ ，但方差很高，导致 RL 不稳定。为了稳定训练、降低方差，人们使用的一种工具是基线（baseline）。

基线可以很复杂，但最简单的基线就是从奖励中减去一个常数 b ：

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)]. \quad (15)$$

例如，如果减去 $b = 0.5$ ，而奖励 r 是二元的（正确为 1，错误为 0），那么使用这一估计器意味着：从模型采样回答，提高正确回答的权重（权重为 0.5），降低错误回答的权重（权重为 -0.5 ）。这与原始梯度估计器形成对比：原始估计器以权重 1 提高正确回答的权重，而对错误回答不做任何处理（权重为 0）。

本节的关键数学结论是：只要基线 b 不依赖动作/回答 y ，减去它就会保持估计器的期望不变。例如， b 可以是常数、 x 的函数，或策略的函数。只要 b 不依赖 y ，在估计器中减去它，就不会改变期望：

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [r(y | x) \nabla_\theta \log \pi_\theta(y | x)]. \quad (16)$$

这一结论直接来自以下恒等式：

$$E_{y \sim \pi_\theta(y | x)} [\nabla_\theta \log \pi_\theta(y | x)] = \nabla_\theta E_{y \sim \pi_\theta(y | x)} [1] = 0, \quad (17)$$

其中第一步是反向应用前面的对数导数技巧。

虽然基线不改变估计器的期望，却可能增大或减小方差。用记号表达，“方差”的含义如下：将策略梯度估计器写成样本均值 $\frac{1}{n} \sum_{i=1}^n Z_i$ ，其中每个 Z_i 是样本 (x, y) 对应的表达式 $(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)$ ，则估计器的方差为 $\frac{E[Z_i^2] - E[Z_i]^2}{n}$ 。下面的题目将探索何时基线会增大或减小方差。

题目 (baseline_calcs)：计算策略梯度估计器的方差 (5 分)

设 π_θ 在二元动作空间 $\mathcal{A} = \{0, 1\}$ 上定义一个策略，且 $\pi_\theta(A = 1) = p = \sigma(\theta)$ ，其中 σ 为 sigmoid 函数 $\sigma(\theta) = \frac{1}{1 + e^{-\theta}}$ 。二元奖励函数对动作 $A = 1$ 赋予奖励 1，其他情况下赋予奖励 0，即 $r(A) = \mathbb{1}\{A = 1\}$ 。

(a) 设策略梯度估计器为

$$\frac{1}{n} \sum_{i=1}^n r(A_i) \nabla_\theta \log \pi_\theta(A_i) \quad (18)$$

给定 n 个样本 $A_i \stackrel{\text{iid}}{\sim} \pi_\theta$ ，该估计器的方差是多少？

提交内容：用 n 和 p 表示的表达式，并附上推导。

(b) 设使用基线调整后的策略梯度估计器为

$$\frac{1}{n} \sum_{i=1}^n (r(A_i) - b) \nabla_\theta \log \pi_\theta(A_i) \quad (19)$$

给定 n 个样本 $A_i \stackrel{\text{iid}}{\sim} \pi_\theta$ ，该估计器的方差是多少？

提交内容：用 n 、 b 和 p 表示的表达式，并附上推导和一些讨论。

(c) 如果代入“总体均值”基线 $b = p$ ，得到的方差是多少？将它与未调整的策略梯度估计器的方差比较：它是始终更低、始终更高，还是会随 p 的不同而有时更高、有时更低？

GRPO 中的基线

GRPO 的第一个组成部分，是使用“组均值”基线：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)}) \quad (20)$$

其中 $\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} | x^{(i)})$ ，即模型在提示 $x^{(i)}$ 上的平均奖励。注意，尽管 μ_i 依赖回答 y ，它仍然在相差缩放因子 $\frac{G-1}{G}$ 的意义下保持期望：

$$E_{y^{(1)}, \dots, y^{(G)} \sim \pi_\theta(y | x)} \left[\frac{1}{G} \sum_{j=1}^G (r(y^{(j)} | x) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(j)} | x) \right] \quad (21)$$

$$= \frac{1}{G} \sum_{j=1}^G E_{y^{(2)}, \dots, y^{(G)} \sim \pi_\theta(y | x)} \left[E_{y^{(1)} \sim \pi_\theta(y | x)} [(r(y^{(1)} | x) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(1)} | x)] \right] \quad (22)$$

$$= \frac{1}{G} \sum_{j=1}^G E_{y^{(1)} \sim \pi_\theta(y | x)} \left[\left(r(y^{(1)} | x) - \frac{1}{G} r(y^{(1)} | x) \right) \nabla_\theta \log \pi_\theta(y^{(1)} | x) \right] \quad (23)$$

$$= \frac{G-1}{G} E_{y \sim \pi_\theta(y | x)} [r(y | x) \nabla_\theta \log \pi_\theta(y | x)]. \quad (24)$$

经过组均值调整后的奖励通常称为优势 (advantage)：它不再表示绝对奖励，而是表示这个 rollout 相对于平均奖励的“优势”。

4.1.6 优势归一化

GRPO 的下一项修改是在减去组均值之后，再除以组标准差：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)}), \quad (25)$$

其中 $\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i)^2}$ （注意，默认的 `torch.std` 函数使用略有不同的样本标准差表达式，以进行偏差校正；实现时应使用默认的 `torch.std`）。除以标准差不再保持估计器的期望，因此我们不再对原始目标 J_θ （期望奖励）做梯度上升。一种理解方式是把它视为稳定性技巧：如果假设各个梯度向量是独立同分布的高斯随机变量，

那么除以组标准差，可以确保各组梯度更新的范数大致相等。下一节会更深入地探讨优势归一化。

4.1.7 序列归一化

GRPO 还有一个似乎继承自早期 LLM-PPO 实现的细节，称为序列归一化。回顾到目前为止的策略梯度估计器如下（这里把 $\log \pi_\theta(y^{(i,j)} | x^{(i)})$ 展开为对时间步的求和）：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (26)$$

GRPO 额外加入一个按序列长度归一化的项：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \left(\frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right). \quad (27)$$

注意，这项修改进一步改变了期望：不同序列中的 token 不再具有相同权重，较长序列中的 token 相对较短序列中的 token 被降低了权重。下一节中，我们将讨论这一选择，并通过消融实验检查它是否是有益的修改。不过，我们先按照原始论文实现标准 GRPO，因此会保留序列归一化。

4.1.8 整合所有组成部分

GRPO 还有一个尚未讨论的组成部分，即带裁剪的重要性重加权。我们会在本作业后面的异策略 RL 部分介绍它。简而言之，异策略 RL 是指对每个推理批次执行多次梯度更新，以加速 RL；与之相对，同策略 RL 对每个推理批次只执行一次梯度更新。“异策略”指的是：从第二次梯度更新开始，推理批次中的样本已经变成旧样本，因为它们来自模型的较早版本。重要性重加权是对梯度估计器的一种修改，使其能够使用这些旧样本。目前我们只做同策略 RL，因此暂时不需要重要性重加权。

现在，我们已经具备实现同策略 GRPO 的全部组成部分。算法见算法 1。

算法 1: 同策略组相对策略优化 (GRPO)

输入 初始策略模型 π_{θ_0} ; 奖励函数 r ; 任务分布 (或数据集) ρ ; 学习率 α 。

输出 π_{θ}

1 策略模型 $\pi_{\theta} \leftarrow \pi_{\theta_0}$

2 对 $\text{step} = 1, \dots, \text{n_grpo_steps}$:

3 采样一批问题 $x^{(1)}, \dots, x^{(B)} \stackrel{\text{iid}}{\sim} \rho$ 。

4 为每个问题采样 G 个输出: $y^{(i,1)}, \dots, y^{(i,G)} \stackrel{\text{iid}}{\sim} \pi_{\theta}(y \mid x^{(i)})$ 。

5 计算同策略 GRPO 策略梯度估计器:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}), \quad (28)$$

其中,

$$\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} \mid x^{(i)}) \quad (29)$$

表示组均值, 而

$$\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} \mid x^{(i)}) - \mu_i)^2} \quad (30)$$

表示组标准差。

6 用你喜欢的优化器更新 θ (下面给出随机梯度上升更新):

$$\theta \leftarrow \theta + \alpha \hat{g}. \quad (31)$$

4.2 实现同策略 GRPO

4.2.1 使用 Hugging Face 模型

前几次作业使用的是我们在 `cs336_basics` 中自行实现的语言模型; 本次作业将直接使用 Hugging Face 的 `transformers` 库来加载预训练基础模型。你也可以自行实现 Transformer 和预训练权重加载器, 只需确保架构与 OLMo-2-0425-1B 一致。要加载 Hugging Face 模型和分词器 (使用 `bfloat16` 与 `FlashAttention-2` 来节省显存), 可使用以下起始代码, 它位于 `cs336_alignment/checkpoint.py`:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device=='cpu' else "flash_attention_2",
    )
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
return model, tokenizer
```

`model_id_or_directory` 可以是模型名，例如 `allenai/OLMo-2-0425-1B`，也可以是目录路径。该目录通常由 `save_pretrained` 生成；你可以调用它来保存训练后的模型：

```
# 保存模型权重
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

首先，实现一个辅助函数，使用预训练分词器对输入提示与回答进行分词。应分别对提示和回答分词，不添加特殊 token，然后直接通过 `prompt_ids + response_ids` 拼接，中间不插入 EOS、BOS 或分隔 token。还需要构造 `response_mask`：这是一个与移位后的 labels 对齐的布尔掩码，标签 token 来自回答的位置恰好为 `True`，提示和填充位置为 `False`。训练循环将使用该掩码，确保仅在回答 token 上计算损失。

题目 (`tokenize_prompt_and_output`)：提示与输出的分词（1 分）

提交内容：实现方法 `tokenize_prompt_and_output`：分别对提示与输出分词，在不插入特殊 token 的情况下拼接，并构造 `response_mask`。推荐接口如下：

```
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizer,
) -> dict[str, torch.Tensor]:
```

对提示字符串与输出字符串分词，并构造与 labels 对齐的掩码：回答 token 为 1，其他 token（提示或填充）为 0。

参数：

- `prompt_strs: list[str]`：提示字符串列表。
- `output_strs: list[str]`：输出字符串列表。
- `tokenizer: PreTrainedTokenizer`：用于分词的分词器。

返回：

- `dict[str, torch.Tensor]`。令 `prompt_and_output_lens` 为分词后拼接的提示与输出字符串长度所组成的列表。返回字典应包含以下键：
 - `input_ids`：形状为 `(batch_size, max(prompt_and_output_lens) - 1)` 的 `torch.Tensor`，即分词并拼接的提示与输出，去掉最后一个 token。
 - `labels`：形状为 `(batch_size, max(prompt_and_output_lens) - 1)` 的 `torch.Tensor`，即移位后的输入 ID，也就是去掉第一个 token 的输入 ID。

- `response_mask`: 形状为 $(\text{batch_size}, \max(\text{prompt_and_output_lens}) - 1)$ 的 `torch.Tensor`; 该掩码与 `labels` 对齐, 对应标签 token 属于回答时为 1, 否则为 0。

要测试代码, 请实现 `[adapters.run_tokenize_prompt_and_output]`。然后运行 `uv run pytest -k test_tokenize_prompt_and_output`, 确保实现通过测试。

拿到分词后的输入, 就可以按如下方式将其传入模型:

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

掌握这些调用方式后, 请实现下面的方法, 计算模型对回答中每个 token 给出的对数概率。这是计算策略梯度所需的基本操作。在 RL 中, 记录逐 token 熵通常也很有用, 因此该函数包含 `return_token_entropy` 选项, 你也需要实现它。

题目 (`get_response_log_probs`): 回答的对数概率 (与熵) (1 分)

提交内容: 实现方法 `get_response_log_probs`, 从因果语言模型获取逐 token 的条件对数概率 (条件为之前的 token), 并可选返回模型下一个 token 分布的熵。

推荐接口如下:

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数:

- `model: PreTrainedModel`: 用于打分的 Hugging Face 模型; 应放在正确的设备上。如果不需要计算梯度, 则置于推理模式。
- `input_ids: torch.Tensor`: 形状为 $(\text{batch_size}, \text{sequence_length})$; 由你的分词方法产生的“提示 + 回答”拼接 token。
- `labels: torch.Tensor`: 形状为 $(\text{batch_size}, \text{sequence_length})$; 由你的分词方法产生的标签。
- `return_token_entropy: bool`: 若为 `True`, 还需返回逐 token 熵。

返回:

- `dict[str, torch.Tensor]`。
 - `"log_probs"`: 形状为 $(\text{batch_size}, \text{sequence_length})$, 条件对数概率 $\log p_{\theta}(x_t | x_{<t})$ 。
 - `"token_entropy"`: 可选; 形状为 $(\text{batch_size}, \text{sequence_length})$, 每个位置的逐 token 熵 (仅在 `return_token_entropy=True` 时存在)。

要测试代码，请实现 `[adapters.run_get_response_log_probs]`。然后运行 `uv run pytest -k test_get_response_log_probs`，确保通过测试。

4.2.2 在强化学习循环中使用 vLLM

在 RL 循环中，我们还需要生成 rollout。具体配置为：将 Hugging Face 模型与优化器放在一张 GPU 上进行训练，将 vLLM（包含模型和 KV 缓存）放在另一张 GPU 上。因此，除了前一节介绍的 vLLM 初始化和生成函数，还需要在每次推理之前同步两张设备之间的权重。下面介绍权重同步代码，它位于 `cs336_alignment/vllm_utils.py`。

```
@dataclass
class VLLMServer:
    gpu: int = 1 # 在 GPU 0 上训练，在 GPU 1 上推理

    # 在训练 GPU 与 vLLM 之间创建 NCCL 权重传输组。
    def init_weight_sync(self, policy_device: str): ...

    # 将当前 Hugging Face 策略权重复制到 vLLM 服务器，
    # 并重置依赖旧权重的 vLLM 缓存。
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # 使用 vLLM 当前权重生成 rollout。
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

与提示实验相同，采样使用温度 1.0、top-p 1.0、最大生成长度 512。提示要求模型以 `</answer>` 结束答案，因此可以指示 vLLM 在输出该字符串时停止：

```
# 参考 Dr. GRPO：模型完成答案时停止
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

4.2.3 GRPO 的组成部分

接下来，实现用于计算 GRPO 损失各部分的组件。注意，在后续作业中还会实现不同变体，例如不同的优势归一化方法或重要性重加权方法。下面的组件经过设计，可在尽可能少重复代码的情况下切换变体，因此我们提供了指定变体的接口参数。在当前部分，你只需实现标准 GRPO 变体。

第一步是实现一个用于计算回答奖励的辅助函数。

题目 (`compute_rollout_rewards`): 计算 rollout 的奖励 (1 分)

提交内容：实现方法 `compute_rollout_rewards`，为每条 rollout 回答计算原始奖励。

推荐接口如下：

```
def compute_rollout_rewards(
    reward_fn: Callable[[str, str], dict[str, float]],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
) -> tuple[torch.Tensor, dict[str, float]]:
```

计算 rollout 回答列表的奖励，以及奖励组成部分的元数据。

参数：

- `reward_fn: Callable[[str, str], dict[str, float]]`：根据标准答案给 rollout 回答评分，返回含 "reward"、"format_reward" 和 "answer_reward" 键的字典。
- `rollout_responses: list[str]`：策略产生的 rollout。列表长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- `repeated_ground_truths: list[str]`：样本的标准答案。列表长度为 `rollout_batch_size`，因为每个样本的标准答案重复了 `group_size` 次。

返回：

- `tuple[torch.Tensor, dict[str, float]]`。
 - `raw_rewards`：形状为 `(rollout_batch_size,)`；每个 rollout 回答尚未归一化的奖励。
 - `metadata`：需要记录的奖励统计量。至少应包含整个 rollout 批次的平均总奖励与平均格式奖励。

要测试代码，请实现 `[adapters.run_compute_rollout_rewards]`。然后运行 `uv run pytest -k compute_rollout_rewards`，确保通过测试。

接下来实现 GRPO 的核心数学部分，即把奖励归一化为优势。注意，上述函数输出的奖励已被展平，因此需要传入组大小，将其重塑为分组形式，再进行归一化。要通过测试，需要支持 `baseline = "mean"` 与 `advantage_normalizer = "std"`。为避免除以零，除法之前还应在每组标准差上加 `advantage_eps`。

题目 (`compute_group_normalized_rewards_grpo`)：组归一化 (1 分)

提交内容：实现方法 `compute_group_normalized_rewards`，在每组内归一化原始奖励，并返回归一化奖励及你认为有用的元数据。

目前只需支持 `baseline = "mean"` 和 `advantage_normalizer = "std"`。对于不支持的输入，可以抛出 `NotImplementedError`。在作业后续部分，我们将实现其他选项并运行消融实验。记得在归一化分母中加入 `advantage_eps`，避免除以零。推荐接口如下：

```
def compute_group_normalized_rewards(
    raw_rewards: torch.Tensor,
    group_size: int,
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
):
```

在每组内应用指定的基线与归一化方法，计算优势。

参数：

- `raw_rewards: torch.Tensor`：形状为 $(\text{rollout_batch_size},)$ ；每个 rollout 回答未归一化的奖励，其中 $\text{rollout_batch_size} = \text{n_prompts_per_rollout_batch} * \text{group_size}$ 。
- `group_size: int`：每个问题（组）的回答数量。
- `baseline: Literal["mean", "none"]`：本题需支持 `mean`，即减去每组平均奖励。后续的 `none` 表示不减去基线。
- `advantage_eps: float`：用于避免归一化时除以零的小常数。
- `advantage_normalizer: Literal["std", "none", "mean"]`：本题需支持 `std`，即除以每组标准差。后续的 `none` 表示不归一化，`mean` 表示除以每组平均奖励。

返回：

- `tuple[torch.Tensor, dict[str, float]]`。
 - `advantages`：形状为 $(\text{rollout_batch_size},)$ ；各 rollout 回答经组归一化后的奖励。
 - `metadata`：自行选择的其他日志统计量，例如奖励的均值、标准差、最大值／最小值。

要测试代码，请实现 `[adapters.run_compute_group_normalized_rewards]`。然后运行 `uv run pytest -k compute_group_normalized_rewards_grpo`，确保通过测试。

下一步是实现一个计算逐 token 策略梯度损失的函数。注意：算法 1 中的公式给出的是梯度估计器，而本题要求你写出一个逐 token 损失，使得对它求梯度时，得到 GRPO 梯度估计器的各项（随后再对 token 聚合）。这个表达式并不是传统意义上那种预期会在训练中下降的损失。它只是一个求导后能得到策略梯度的表达式。

具体需要实现的损失为：

$$J_{\theta}^{\text{GRPO-on-policy}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}), \quad (32)$$

你可以验证，对其求导会得到算法 1 中的梯度项。由于 PyTorch 优化器执行梯度下降，实现应返回该目标的负值作为损失。`compute_policy_gradient_loss` 计算每个序列中逐 token 的损失项，`aggregate_loss_across_microbatch_sequence` 则在 token 和序列之间对逐 token 损失求平均。

题目 (compute_policy_gradient_loss_on_policy): 同策略策略梯度 (1 分)

提交内容: 实现方法 `compute_policy_gradient_loss`, 根据原始奖励或预先计算的优势, 计算逐 token 的策略梯度损失。目前假设所有 rollout 都来自同策略, 因此只需支持 `importance_reweighting_method = "none"`, 可以忽略旧对数概率和裁剪参数。对于不支持的输入, 可以抛出 `NotImplementedError`。裁剪将在作业后面实现。

推荐接口如下:

```
def compute_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    response_mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

计算每个 token 上的策略梯度损失, 其中 `raw_rewards_or_advantages` 可以是原始奖励, 也可以是已经归一化的优势。

参数:

- `raw_rewards_or_advantages: torch.Tensor`: 形状为 `(batch_size,)` 或 `(batch_size, 1)`, 每个 rollout 回答对应的标量奖励/优势。
- `policy_log_probs: torch.Tensor`: 形状为 `(batch_size, sequence_length)`, 每个 token 的对数概率。
- `importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"]`: "none" 表示不进行重要性重加权; "noclip" 表示重要性重加权但不裁剪; "grpo" 表示 PPO / GRPO 风格的 token 级重加权和裁剪; "gspe" 表示 GSPO 风格的序列级重加权和裁剪。
- `old_log_probs: torch.Tensor | None`: 除 `importance_reweighting_method = "none"` 外均为必需; 形状为 `(batch_size, sequence_length)`。
- `cliprange: float | None = None`: 裁剪参数 ϵ ; `importance_reweighting_method` 为 "grpo" 或 "gspe" 时必需。
- `response_mask: torch.Tensor | None = None`: 可选的回答 token 掩码, 形状为 `(batch_size, sequence_length)`。对于仅在回答 token 上计算序列级对数比值平均值的 GSPO 实现, 该参数是必需的。

返回:

- `tuple[torch.Tensor, dict[str, torch.Tensor]]`.
 - `per_token_policy_gradient_loss`: 形状为 `(batch_size, sequence_length)`, 逐 token 的策略梯度损失 (将在训练循环中沿批次维和序列维聚合)。
 - `metadata`: 底层损失函数调用产生的统计量, 例如裁剪比例的组成项。

要测试代码，请实现 `[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_on_policy`，确保通过测试。

最后，实现跨 token 与序列的聚合函数。标准 GRPO 先对每条序列中的 token 求平均，再对序列求平均。后面我们将实现另一种做法：只除以一个常数。正如推导中所见，这更忠实于最初希望估计的策略梯度。

题目 (`aggregate_loss_across_microbatch_sequence`): 跨 token 与序列聚合损失 (0.5 分)

提交内容: 实现方法 `aggregate_loss_across_microbatch`，接收逐 token 的策略梯度损失与回答掩码，输出平均损失。目前采用标准 GRPO 聚合，即先在每个序列内求平均，再对序列求平均，因此可以假设 `loss_normalization = "sequence"`。对于不支持的输入，可以抛出 `NotImplementedError`。

推荐接口如下：

```
def aggregate_loss_across_microbatch(
    per_token_policy_gradient_loss: torch.Tensor,
    mask: torch.Tensor,
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> torch.Tensor:
```

按照回答掩码与损失归一化策略，聚合逐 token 的策略梯度损失。

参数：

- `per_token_policy_gradient_loss: torch.Tensor`: 形状为 `(batch_size, sequence_length)`，逐 token 策略梯度损失（将在训练循环中沿批次维与序列维聚合）。
- `mask`: 形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`，表示哪些位置应计入损失。
- `loss_normalization: Literal["sequence", "constant"] = "sequence"`: "sequence" 表示先对每条序列的损失求平均，再对序列求平均；"constant" 表示用一个常数归一化总损失。
- `normalization_constant: int | None = None`: 总损失要除以的常数；当 `loss_normalization = "constant"` 时必需。

返回：

- `loss: torch.Tensor`: 包含平均损失的标量。确保后续可以对这个损失调用 `backward`。

要测试代码，请实现 `[adapters.run_aggregate_loss_across_microbatch]`。然后运行 `uv run pytest -k test_aggregate_loss_across_microbatch_sequence`，确保通过测试。

4.2.4 GRPO 训练步

现在可以把这些组件组合为一个完整训练步。

梯度累积

为了在推理过程中获得较高的利用率，需要较大的批次。因此，在训练批次大小与推理批次大小相等的同策略 RL 中，GPU 显存不足以一次计算整个批次的梯度。我们需要把批次分成一系列微批次（microbatch），并跨微批次累积梯度。主要的难点是正确处理归一化，确保按微批次累积得到的梯度，与一次对整个批次计算的梯度等价。

在 PyTorch 中实现梯度累积很直接。回顾一下，每个权重张量都具有存储梯度的 `.grad` 属性。调用 `loss.backward()` 之前，`.grad` 为 `None`；调用之后，`.grad` 中保存梯度。通常，我们会执行一次优化器更新，再用 `optimizer.zero_grad()` 将梯度清零，该调用会重置权重张量的 `.grad` 字段：

```
# 前向传播。
logits = model(inputs)
loss = loss_fn(logits, labels)
# 反向传播。
loss.backward()
# 更新权重。
optimizer.step()
# 清零梯度，为下一轮迭代做准备。
optimizer.zero_grad()
```

实现梯度累积时，将批次分成 k 个微批次，只调用一次 `optimizer.step()` 和 `optimizer.zero_grad()`。假设使用序列归一化，即先在各序列内求损失平均值，再跨序列平均，那么计算出微批次平均损失后，还需要根据序列数量重新加权。

```
gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps
for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i+microbatch_size]
    labels_microbatch = labels[i:i+microbatch_size]
    # 前向传播。
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (len(inputs_microbatch) / len(inputs))
    # 反向传播。
    loss.backward()
    # 对整个批次只更新一次权重。
    optimizer.step()
    # 对整个批次只清零一次梯度。
    optimizer.zero_grad()
```

注意，后续使用常数归一化代替序列归一化时，微批次代码也需要略作修改，以适应不同的归一化方式。

实现训练步

在下面的函数中，我们最终将给定的一批 rollout 转换为一次 GRPO 训练步。函数接收模型、分词器、优化器、奖励函数、提示和 rollout，以及各种超参数，累积梯度后执行一次优化器更新。为避免显存不足，需要按上文实现梯度累积。优化器更新之前，还应将梯度范数裁剪到 `max_grad_norm`。随后函数应返回该批次的训练损失及元数据，供日志记录使用。训练步至少需要记录：

- 损失。
- 梯度范数。
- token 熵。
- 训练奖励（总奖励、格式奖励）。

也可以记录其他指标，例如对从 1 到 `group_size` 的各个 k 记录 `pass@k`（即前 k 个 rollout 中至少有一个正确答案的问题所占比例），这可能很有用。

题目 (`grpo_train_step_standard_on_policy`): GRPO 训练步 (5 分)

提交内容：给定模型、分词器和 rollout，实现一次策略梯度批次更新。本部分只需实现同策略设置下的标准 GRPO，即 `baseline = "mean"`、`advantage_normalizer = "std"`、`importance_reweighting_method = "none"`、`loss_normalization = "sequence"`。对于不支持的输入，可以抛出 `NotImplementedError`。

推荐接口如下：

```
def grpo_train_step(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizer,
    optimizer: Optimizer,
    gradient_accumulation_steps: int,
    max_grad_norm: float | None,
    reward_fn: Callable[[str, str], dict[str, float]],
    repeated_prompts: list[str],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    # 奖励归一化
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
    # 重要性重加权与裁剪
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    # 损失归一化
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

使用 `gradient_accumulation_steps` 个微批次，执行前向与反向传播。

参数：

- `model`: `PreTrainedModel`: 需要训练的 Hugging Face 模型。
- `tokenizer`: `PreTrainedTokenizer`: 用于分词的分词器。
- `optimizer`: `Optimizer`: 模型的优化器。
- `gradient_accumulation_steps`: `int`: 每次优化器更新包含的微批次数量。
- `max_grad_norm`: `float | None`: 若不为 `None`，则在调用 `optimizer.step()` 之前，将梯度范数裁剪到该值。
- `reward_fn`: `Callable[[str, str], dict[str, float]]`: 根据标准答案对 rollout 回答评分，返回含 `"reward"`、`"format_reward"` 与 `"answer_reward"` 键的字典。
- `repeated_prompts`: `list[str]`: 各样本的提示。列表长度为 `rollout_batch_size`，因为每个样本的提示重复 `group_size` 次。
- `rollout_responses`: `list[str]`: 策略产生的 rollout。列表长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- `repeated_ground_truths`: `list[str]`: 样本的标准答案。列表长度为 `rollout_batch_size`，因为每个样本的标准答案重复 `group_size` 次。
- `group_size`: `int`: 每个问题（组）的回答数量。
- `baseline`: `Literal["mean", "none"]`: 若为 `mean`，减去每组平均奖励；若为 `none`，不做处理。
- `advantage_eps`: `float`: 避免归一化时除以零的小常数。
- `advantage_normalizer`: `Literal["std", "none", "mean"]`: 若为 `std`，除以每组标准差；若为 `none`，不做处理；若为 `mean`，除以每组平均奖励。
- `importance_reweighting_method`: `Literal["none", "noclip", "grpo", "gspo"]`: `"none"` 表示不进行重要性重加权；`"noclip"` 表示重要性重加权但不裁剪；`"grpo"` 表示 PPO / GRPO 风格的 token 级重加权和裁剪；`"gspo"` 表示 GSPO 风格的序列级重加权和裁剪。
- `old_log_probs`: `torch.Tensor | None`: 除 `importance_reweighting_method = "none"` 外均为必需；形状为 `(batch_size, sequence_length)`。
- `cliprange`: `float | None = None`: 裁剪参数 ϵ ；`importance_reweighting_method` 为 `"grpo"` 或 `"gspo"` 时必需。
- `loss_normalization`: `Literal["sequence", "constant"] = "sequence"`: `"sequence"` 表示先对各序列内的损失求平均，再跨序列平均；`"constant"` 表示用一个常数（整个训练期间固定）归一化总损失。
- `normalization_constant`: `int | None = None`: 总损失要除以的常数；当 `loss_normalization = "constant"` 时必需。

返回：

- `tuple[torch.Tensor, dict[str, torch.Tensor]]`.
 - `loss`: 标量张量；已针对梯度累积作过调整的批次损失。返回它是为了记录日志。
 - `metadata`: 字典，包含底层损失调用的元数据、裁剪前的梯度范数，以及其他你希望记录的统计量。

要测试代码，请实现 [adapters.run_grpo_train_step]。然后运行 `uv run pytest -k test_grpo_train_step_standard_on_policy`，确认通过测试。

4.3 实验

现在可以构建完整的 GRPO 训练循环。它应当：

- 加载模型和数据集；
- 初始化日志记录（例如 Wandb）、vLLM 服务器和优化器；
- 执行 RL 训练循环。

下面给出一些建议超参数。如果训练脚本正确，在大多数随机种子下使用这些超参数训练时，应该能看到验证奖励随训练逐步提升。

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0
optimizer = torch.optim.AdamW(
    policy.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.0
)
```

注意，`rollout_batch_size` 与 `train_batch_size` 统计的是回答数，而不是提示数。因此，`rollout_batch_size = train_batch_size = 256` 表示 32 个提示，每个提示有 8 个 rollout。

一个合理的默认设置是每生成 10 个 rollout 批次，就在验证集上评估一次。应确保至少使用 1024 个验证样本，因为 CoT / RL 的评估可能存在较大噪声。除了上一小节描述的指标，记录 rollout 也有助于定性理解模型正在做什么。合理的默认设置是每 40 个 rollout 批次，记录当前批次的训练 rollout。训练样本可以使用 `data/gsm8k/train.jsonl`，验证样本可以使用 `data/gsm8k/test.jsonl`。

由于策略梯度估计器可能具有很高方差，且 RL 会自我强化，不同 RL 训练运行的轨迹往往具有非常大的方差。因此，一旦你确认脚本正确，就需要使用 4 个随机种子运行 RL 循环。在固定种子下，脚本不必完全可复现；也可以直接将实验运行 4 次。在绘图时，请用某种方式展示不同运行之间的方差，不要只画均值。一些合理选项包括：

- 绘制 4 次运行的均值，同时直接绘制各次实际运行曲线。为了可读性，可以把实际运行曲线放在另一张图中，或使用更低透明度、更细线条、虚线。
- 绘制均值与阴影置信区间：具体来说，对每一步计算样本均值和样本标准差。一个合理区间是 $\mu \pm 1.96 \cdot \frac{\sigma}{\sqrt{n}}$ ，其中 n 为种子数量。

- 绘制均值，以及每一步的最小值与最大值。

题目 (grpo_experiments_standard_on_policy)：使用 GRPO 提升 OLMo-2-0425-1B 在 GSM8K 上的性能（2 B200 小时；10 分）

(a) 编写脚本，在给定模型名称、提示、训练集文件路径、验证集文件路径、采样超参数和训练超参数时运行 GRPO 训练循环。从高层次看，脚本首先初始化 vLLM 服务器、wandb 日志、数据集、模型和优化器；随后在训练循环中反复执行：与 vLLM 服务器同步权重、生成训练 rollout、在训练批次上计算策略梯度，并定期检查验证集性能，同时记录生成结果。

提交内容：在 GSM8K 任务和 OLMo-2-0425-1B 模型上运行标准同策略 GRPO 的脚本。

(b) 运行脚本约 50 步，确认验证奖励有所提升，而且随训练产生的 rollout 看起来合理。注意：奖励从接近零开始是正常现象；根据随机种子的不同，可能需要几步才能采样到非零奖励的 rollout，模型才会开始改善。

提交内容：能够让你确信脚本正确的证据，例如验证奖励逐步改善、合理的 rollout。

(c) 确认脚本正确后，使用上面提供的超参数，在 GSM8K 上对 OLMo-2-0425-1B 运行实验，共使用 4 个随机种子。应使用零样本 `r1_zero` 提示。代码正确时，应该看到模型随着训练，验证奖励逐步提高。

记录以下指标，绘制它们随训练变化的曲线，并展示运行之间的方差，例如绘制各次运行的均值／标准差或最小值／最大值：

- 损失。
- 梯度范数。
- token 熵。
- 训练奖励（总奖励、格式奖励）。
- 验证奖励（总奖励、格式奖励）。
- 验证集平均回答长度。
- 其他你认为有助于调试的内容。

还请定期记录并观察 rollout。训练是否使回答得到改善？

提交内容：用几句话评论，并为每项指标提供一张图，描述它在训练中的变化以及运行间方差的大小。

提交内容：给出若干训练前后的模型 rollout 示例。

提交内容：给出一套能够达到最终验证准确率至少 25% 的流程，准确率按随机种子取平均。

现在已有可运行的脚本，可以开始调节超参数并开展消融实验。首先调节学习率，因为它通常是对训练影响最大的超参数。

题目 (grpo_learning_rate): 调节学习率 (4 B200 小时; 3 分)

从建议超参数出发，对学习率进行扫描实验：至少包括一个低于默认建议值的学习率，以及一个高于默认值的学习率。报告最终验证奖励；若优化器发散，则注明发散。根据上一部分观察到的方差，决定每组训练应使用多少个随机种子。在作业剩余部分，可以使用调优后的学习率代替建议默认值。

提交内容：一张“最终验证奖励—学习率”曲线图，以及几句评论。

另一个会影响 RL 动态的选择是提示：由于 RL 会提高模型 rollout 的权重，提示会塑造训练过程中模型所探索的 rollout。下面探索提示选择的影响。

题目 (grpo_prompt_ablation): 提示消融 (4 B200 小时; 3 分)

使用若干随机种子，分别以 question_only 提示和 r1_zero_three_shot 提示运行 GRPO 脚本。与前面零样本 r1_zero 的结果相比，这些提示表现如何？哪些提示的平均奖励最高、方差最低？其他已记录指标中是否存在系统性差异？根据运行间的方差，你对结论有多大把握？

提交内容：评论，并以相应指标的图表支持论述。

5 RL 算法变体

GRPO 是语言模型 RL 中的常见选择，但不同论文对其算法设计选择存在争议。本作业将学习这些选择背后的一些理论论点，并通过受控实验亲自观察哪些算法更好，至少对我们使用的模型—数据集组合而言如此。本节讨论同策略设置，下一节讨论异策略算法。

5.1 Dr. GRPO

在 GRPO 的推导中，你可能已经注意到，一系列选择使 GRPO 策略梯度估计器不再具有“正确”的期望 $\nabla_{\theta} J_{\theta}$ ，其中 J_{θ} 为模型的期望奖励。这两个选择分别是：使用标准差对优势归一化，以及按序列长度归一化。我们首先考虑 Dr. GRPO 论文所主张的消融 [Z. Liu 等, 2025]，即撤销这两个选择：去掉标准差归一化，并用常数除以总损失，而不是先按各序列长度归一化。将该归一化常数记为 Z ，Dr. GRPO 梯度估计器为：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} (r(y^{(i,j)} | x^{(i)}) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}), \quad (33)$$

其中, Z 通常设为训练批次大小乘最大生成长度, 即 $Z = BGL$, L 是最大生成长度, 在本作业中为 512。

题目 (think_about_length_normalization): 思考长度归一化 (1 分)

在运行任何实验之前, 思考以下两种方式的区别: 按每条序列各自的长度归一化, 或者用同一个常数归一化所有序列。它们各有什么优缺点? 是否存在某些具体设置或示例, 使其中一种看起来更好?

提交内容: 几句话的讨论。

题目 (compute_group_normalized_rewards_drgrp): Dr. GRPO 组归一化 (0.5 分)

提交内容: 更新方法 `compute_group_normalized_rewards`, 支持 `advantage_normalizer = "none"`。虽然 Dr. GRPO 使用 `baseline = "mean"`, 本题也请加入对 `baseline = "none"` 的支持。

要测试代码, 请实现 `[adapters.run_compute_group_normalized_rewards]`。然后运行 `uv run pytest -k compute_group_normalized_rewards_drgrp`, 确保通过测试。

题目 (aggregate_loss_across_microbatch_constant): Dr. GRPO 损失聚合 (0.5 分)

提交内容: 更新方法 `aggregate_loss_across_microbatch`, 支持 `loss_normalization = "constant"`。

要测试代码, 请实现 `[adapters.run_aggregate_loss_across_microbatch]`。然后运行 `uv run pytest -k test_aggregate_loss_across_microbatch_constant`, 确保通过测试。

5.2 拒绝采样微调

接下来考察一种简单得多的算法, 通常称为“拒绝采样微调” (rejection fine tuning, RFT) 或“专家迭代” (expert iteration, EI)。顾名思义, 算法先采样一批 rollout, 只保留正确的, 再对这些正确 rollout 做监督微调, 即最小化下一个词预测的对数损失。具体地, RFT 梯度为:

$$\hat{g} \leftarrow \nabla_{\theta} \left[\frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \mathbb{1}\{r(y^{(i,j)} | x^{(i)}) = 1\} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right], \quad (34)$$

其中, Z 是常数归一化因子, 与 Dr. GRPO 一样; $\mathbb{1}\{r(y^{(i,j)} | x^{(i)}) = 1\}$ 是仅保留正确回答的指示函数。下面的题目将思考: RFT 梯度是否真的是策略梯度, 即它是否真的在优化期望奖励 J_{θ} , 以及它与 GRPO 有什么关系。和前面一样, 将它实现为 PyTorch 损失时, 应返回目标的负值, 使最小化损失对应于沿估计器做梯度上升。

题目 (think_about_rft): 思考 RFT (2 分)

回顾采用常数归一化的 RFT 目标:

$$J_{\theta} = \frac{1}{Z} \sum_x \sum_{j=1}^G \mathbb{1}\{r(y^{(j)} | x) = 1\} \log \pi_{\theta}(y^{(j)} | x), \quad (35)$$

其中, x 为提示, 每个 $y^{(j)}$ 表示回答 $y^{(j)} \stackrel{\text{iid}}{\sim} \pi_{\theta}(\cdot | x)$, G 为从策略采样的生成结果数量, Z 为常数归一化因子, r 为奖励函数。RFT 梯度即为 $\nabla_{\theta} J_{\theta}$ 。

相比之下, 同策略 Dr. GRPO 的策略梯度估计器, 即采用常数归一化且不做标准差归一化的 GRPO, 为:

$$\frac{1}{Z} \sum_x \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x), \quad (36)$$

其中, μ 为组均值, $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$ 。假设奖励函数 r 是二元的, 比较这两个目标的异同。它们的期望是否相同? 你预期哪一种方差更低? 直观上, 是否存在某些情形, 让你更倾向选择其中之一?

提交内容: 几句话的讨论。

5.3 MaxRL

最后, 我们将探索近期提出的最大似然强化学习 (Maximum Likelihood Reinforcement Learning), 也称 MaxRL [F. Tajwar 等, 2026]。它不像 GRPO 那样除以组标准差, 也不像 Dr. GRPO 那样不归一化, 而是提出将优势除以组均值。由此得到以下梯度估计器:

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\mu_i} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (37)$$

注意, 原始 MaxRL 论文并未使用常数归一化因子 Z , 而是除以批次中的总 token 数 $\sum_{i=1}^B \sum_{j=1}^G \text{len}(y^{(i,j)})$; 这个分母依赖批次, 不再是常数。本作业改为使用常数归一化, 以减少相对于其他基线发生变化的部分, 更容易单独考察以 μ_i 而不是 std_i 归一化的影响。

下面的题目将通过一些基本数学分析, 探索不同优势归一化因子的作用。直观上, 除以平均奖励 μ_i , 会在平均意义上提高更难提示的权重, 因为与简单提示相比, 它们更可能具有较低的 μ_i 。可以定义期望奖励的重加权版本, 使这一直觉形式化:

$$J_{\theta, w} = E_{x \sim \rho} [w(x, \text{stopgrad}(\pi_{\theta})) E_{y \sim \pi_{\theta}} r(y | x)], \quad (38)$$

其中, $w(x, \text{stopgrad}(\pi_{\theta}))$ 对提示 x 重新加权, 例如依据其难度 $\eta(x) = E_{y \sim \pi_{\theta}} r(y | x)$; 这一难度由当前策略在 x 上的期望奖励刻画。 stopgrad 表示不通过重加权函数求梯度。下面的题目将推导此前各个优势归一化因子对应的难度重加权形式。

题目 (derive_difficulty_reweightings): 推导不同优势归一化因子引入的难度重加权 (6 分)

回顾一下，对提示进行总体聚合后的标准策略梯度为：

$$\nabla_{\theta} J_{\theta} = \nabla_{\theta} E_{x \sim \rho} [E_{y \sim \pi_{\theta}} r(y | x)], \quad (39)$$

其中， ρ 为提示 x 的分布， π_{θ} 表示策略， $r(y | x)$ 表示 y 是否正确回答了 x 。本题将推导 GRPO 各变体所优化的代理目标，其形式为：

$$\nabla_{\theta} J_{\theta, w} = \nabla_{\theta} E_{x \sim \rho} [w(x, \text{stopgrad}(\pi_{\theta})) E_{y \sim \pi_{\theta}} r(y | x)] \quad (40)$$

其中， w 是某个对提示的重加权函数，并且我们不通过 w 求导。

(a) 回顾 Dr. GRPO 策略梯度估计器为

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right] \quad (41)$$

其中 μ 表示组均值， $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$ 。设常数归一化因子 $Z = G$ ，并令组大小 $G \rightarrow \infty$ 。对于怎样的重加权函数 w ，该估计器等价于优化代理目标 $J_{\theta, w}$ ？

提交内容：用题目参数的某个子集表示的表达式，并用几句话说明理由。

(b) 假设使用常数归一化，GRPO 估计器与 Dr. GRPO 的区别是除以标准差：

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\text{std}} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right], \quad (42)$$

其中，std 表示组标准差。设常数归一化因子 $Z = G$ ，并令组大小 $G \rightarrow \infty$ 。对于怎样的重加权函数 w ，该估计器等价于优化代理目标 $J_{\theta, w}$ ？

提交内容：用题目参数的某个子集表示的表达式，并用几句话说明理由。

(c) MaxRL 则除以组均值，即

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\mu} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (43)$$

设常数归一化因子 $Z = G$ ，并令组大小 $G \rightarrow \infty$ 。对于怎样的重加权函数 w ，该估计器等价于优化代理目标 $J_{\theta, w}$ ？

提交内容：用题目参数的某个子集表示的表达式，并用几句话说明理由。

题目 (think_about_advantage_normalization): 思考优势归一化 (2 分)

在运行任何实验前，思考用组标准差、组均值归一化组内优势，或不做优势归一化之间的区别。各自有什么优缺点？是否存在某些具体设置或示例，让其中一种看起来更好？

提交内容：几句话的讨论。

题目 (`compute_group_normalized_rewards_maxrl`): MaxRL 组归一化 (0.5 分)

提交内容：更新方法 `compute_group_normalized_rewards`，支持 `advantage_normalizer = "mean"`。与 `advantage_normalizer = "std"` 一样，应在归一化分母中加上 `advantage_eps`，避免除以零。

要测试代码，请实现 `[adapters.run_compute_group_normalized_rewards]`。然后运行 `uv run pytest -k compute_group_normalized_rewards_maxrl`，确保通过测试。

5.4 实验

现在可以运行实验，探索这些不同的策略梯度估计器。首先，需要更新 `grpo_train_step` 方法，使其支持这些变体；相关组件已在前面实现。具体设置如下：

- `GRPO_constant`: `baseline = "mean", advantage_normalizer = "std", loss_normalization = "constant"`。
- `Dr_GRPO`: `baseline = "mean", advantage_normalizer = "none", loss_normalization = "constant"`。
- `RFT`: `baseline = "none", advantage_normalizer = "none", loss_normalization = "constant"`。
- `MaxRL` (常数归一化版本): `baseline = "mean", advantage_normalizer = "mean", loss_normalization = "constant"`。

为了加快训练，请注意：一旦计算出各序列的归一化优势，优势为零的序列就不必传入模型，因为它们对梯度的贡献为零。由于奖励是二元的，以下两种情况会出现零优势序列：

- 若 `baseline = "mean"`，一组内所有序列的奖励相同时，这些序列的优势为零。
- 若 `baseline = "none"`，所有零奖励序列的优势均为零。

删去零优势序列后，还可以按相同比例减少 `gradient_accumulation_steps`，即保持 `microbatch_size` 不变。例如，若一半序列的优势为零，可以将梯度累积步数从 k 减为 $k/2$ 。这项优化对包含大量零奖励序列的 RFT 尤其有帮助。不过，需要仔细处理数学与实现，确保删减版本与未删减版本计算出相同梯度。

题目 (`grpo_train_step_variants_on_policy`): GRPO 训练步变体 (2.5 分)

提交内容：更新 `grpo_train_step` 方法，支持全部同策略变体，仍令 `importance_reweighting_method = "none"`。这些选项包括 `baseline: Literal["mean", "none"]`、`advantage_normalizer: Literal["std", "none", "mean"]`、`loss_normalization: Literal["sequence", "constant"]`。此外，为加快训练，方法应避免将零优势序列传入模型。

注意，`baseline = "none"` 时，错误样本的奖励为零，因此其损失权重也为零，不必将它们传入模型。你可以利用这一点调整实现并加速训练。

要测试代码，请实现 `[adapters.run_grpo_train_step]`。然后运行 `uv run pytest -k test_grpo_train_step_variants_on_policy`，确认通过测试。

实现训练步之后，即可运行实验，观察各方法的表现，判断它们是否优于标准 GRPO。应使用与之前运行相同的超参数，以保证可比性。

关于超参数调优的说明：如果目标是判断一种方法是否优于另一种，最严谨的实验方式是分别为每种方法调节超参数。然而，超参数调优成本高，本次作业没有那么多算力。另一种成本较低的方案是先为基线调优，再把这些超参数用于新方法。例如，我们为标准 GRPO 调节学习率，并在接下来的实验中继续使用该学习率。在这种情况下，如果新方法优于基线，仍可以说它更好，因为这个非最优学习率所产生的表现，为新方法经过充分调优后的性能给出了下界。反过来，若新方法表现较差，则不能断言它比基线差，因为还没有为它适当调优超参数。

题目 (`grpo_experiments_variants_on_policy`): 比较不同 RL 算法的性能 (8 B200 小时; 10 分)

保持相对于标准 GRPO 训练运行的超参数不变（学习率由你选择），对以下每个变体使用 4 个随机种子运行实验。应使用零样本 `r1_zero` 提示。

- `GRPO_constant`: 标准 GRPO，但损失聚合改为常数归一化，而不是序列归一化。
- `Dr_GRPO`: 在 `GRPO_constant` 基础上，令 `advantage_normalizer = "none"`。
- `RFT`: 在 `GRPO_constant` 基础上，令 `advantage_normalizer = "none"` 且 `baseline = "none"`。注意，此方法无需将错误样本传入被训练模型，这应能加快训练。
- `MaxRL`: 在 `GRPO_constant` 基础上，令 `advantage_normalizer = "mean"`。注意，原始 MaxRL 实现按每个微批次的 token 数归一化损失，但本作业改用常数归一化。

与先前运行的标准 GRPO 相比，这些方法表现如何？哪些方法的平均性能最好，哪些方法的方差最低？是否有某些方法可能受益于更多超参数调优？根据运行间方差，你对结论有多大把握？

提交内容：评论，并以所讨论指标的图表支持论述。

6 异策略 RL

目前的实验和算法均完全采用同策略：每个梯度估计器都使用直接从待更新模型中采样的样本，即每个推理批次执行一次训练步，训练批次大小与推理批次大小相等。本节将探索异策略 RL，它对每个推理批次执行多次训练步，有望加速训练，但代价是潜在的不稳定性以及更高的算法复杂度。

6.1 重要性重加权

在最基础的策略梯度方法中，我们先采样一批回答，再基于这些回答执行一次大批次梯度更新。但也许可以把这批回答分成一系列小批次（minibatch），对每个小批次执行一次训练步，从而加快模型训练。这种方法称为“异策略 RL”；相对地，原本对每个推理批次只执行一次训练步的方式称为“同策略 RL”。

之所以称为异策略 RL，是因为第一次小批次更新之后，当前策略已不同于推理时使用的策略。这些样本变成了“异策略”或“陈旧”样本：它们并非来自我们希望计算策略梯度的当前策略。将推理策略记为 π_0 、当前策略记为 π_θ ，有：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)] \neq E_{x \sim \rho} E_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (44)$$

左侧是“朴素”异策略估计器：假装回答 y 来自当前策略，并机械地套用相同步骤计算策略梯度。这个不等式说明，朴素估计器不具有正确的期望。

一种校正偏差的方式是重要性重加权，得到以下估计器：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right], \quad (45)$$

其中应用了重要性权重 $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$ 。这些权重提高对当前策略 π_θ 仍然相关的 y 的权重，同时降低那些 $\pi_\theta(y|x)$ 较小的“旧样本”的权重。不难看出，修改后的表达式具有正确的期望：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right] = E_{x \sim \rho} E_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (46)$$

缺点是重要性重加权会增大估计器方差：如果重要性权重 $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$ 可以大到 C ，方差也可能扩大 C 倍。这使朴素重要性重加权无法适用于语言模型，因为其重要性权重是 $\text{len}(y)$ 项的乘积，会随回答长度呈指数变化：

$$\frac{\pi_\theta(y|x)}{\pi_0(y|x)} = \prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_0(y_t|x, y_{<t})}. \quad (47)$$

因此，与具有较高偏差、但不会由重加权额外引入方差的“朴素”未加权异策略估计器相比，这一重加权估计器的偏差为零，但方差很高。接下来介绍的方法将位于二者之间，通过不同启发式方案在偏差与方差之间权衡。

6.2 PPO / GRPO 风格的重要性重加权与裁剪

6.2.1 token 级重加权

标准做法最初由 PPO 提出 [J. Schulman 等, 2017], 也被 GRPO 采用 [Z. Shao 等, 2024]: 进行 token 级重要性重加权, 而不是序列级重加权。该选择得到以下“未裁剪的 token 级重加权”估计器:

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[r(y|x) \sum_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (48)$$

注意, 它不同于前面具有严格依据的序列级估计器; 后者展开时间步后可写为:

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[r(y|x) \left(\prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \right) \sum_{t=1}^{\text{len}(y)} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (49)$$

在深入理解这一修改后的表达式之前, 先注意: 重加权项不再随 $\text{len}(y)$ 指数变化, 因此方差大幅降低。

与前面各节一样, 为了更好地理解这个估计器, 我们将推导它所优化的代理目标 [T. Degris 等, 2013]。具体地, 令 $\tilde{\pi}_t$ 表示这样一个代理策略: 除第 t 步从 π_θ 采样外, 其余所有时间步均从旧策略 π_0 采样:

$$\tilde{\pi}_t(y|x) = \left(\prod_{s=1}^{t-1} \pi_0(y_s | x, y_{<s}) \right) \pi_\theta(y_t | x, y_{<t}) \left(\prod_{s=t+1}^{\text{len}(y)} \pi_0(y_s | x, y_{<s}) \right). \quad (50)$$

假设所有回答的长度相同, 均为 L , 则 token 级重加权实际上优化的是各个 $\tilde{\pi}_t$ 下的期望奖励, 即:

$$J_\theta^{\text{token}} = E_x \left[\sum_{t=1}^L E_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right]. \quad (51)$$

将这一代理目标用重要性重加权项改写, 即可直接得到该结论:

$$\nabla_\theta J_\theta^{\text{token}} = \nabla_\theta E_x \left[\sum_{t=1}^L E_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right] \quad (52)$$

$$= \nabla_\theta E_x \left[\sum_{t=1}^L E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \right] \right] \quad (53)$$

$$= E_x \left[E_{y \sim \pi_0(y|x)} \left[\sum_{t=1}^L \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right] \right], \quad (54)$$

其中, 最后一步使用了本作业反复使用的对数导数技巧。

这个代理目标为 token 级重加权引入的偏差提供了直观解释。首先, 对于第 t 项, 前缀 $y_{<t}$ 来自 π_0 而不是 π_θ , 因此可能无法代表当前模型产生的前缀。其次, 在从当前策略 π_θ 采样 token y_t 之后, 后缀 $y_{>t}$ 来自 π_0 而不是 π_θ 。也就是说, 当目标评估动作 y_t 如何影响后续步骤时, 依据的是旧策略 π_0 的行为, 而不是当前策略 π_θ 的行为。这两种偏差都会被序列级

重加权所校正，因为它会正确地重加权前缀和后缀，但代价是方差增大。直观上， π_θ 距离 π_0 越远，这些偏差就越严重。

题目 (derive_surrogate_objectives)：推导不同重要性重加权方法的代理目标 (2 分)

前面已经看到，token 级重要性重加权优化的是一组代理策略下的期望奖励；这些代理策略只在一个位置使用当前策略，其余位置都从旧策略采样。现在假设改为考虑一种“成对”的重要性重加权方法，其策略梯度估计器为：

$$\sum_{t=1}^{\frac{L}{2}} \frac{\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \nabla_\theta [\log(\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}))] \quad (55)$$

其中， π_θ 为当前策略， π_0 为旧采样策略，且 $y \sim \pi_0(y | x)$ 。这个估计器在优化什么代理目标？

提交内容：用题目参数的某个子集表示的表达式，并附上推导。

6.2.2 裁剪

除 token 级重加权外，PPO 和 GRPO 还引入了另一种启发式技巧：重要性权重裁剪。正如前面所见， π_θ 与 π_0 相距越远，重要性重加权的方差问题就越严重。代理目标也说明，token 级重加权的偏差会随之加重。因此，为了使异策略 RL 稳定，需要确保 π_θ 与 π_0 保持接近。

最简单的方式是减少每个推理批次上的训练步数，但某些推理批次可能允许执行更多步。为了尽量提高算法性能，我们希望在每个推理批次允许的范围内执行尽可能多的更新。为此，一类方法会裁剪重要性权重过大或过小的项。

近期论文对裁剪的具体实现细节有分歧，但本作业将实现 PPO 中的裁剪方式；该方法一直沿用到 GRPO，而且似乎是主流做法。首先，将 token 级重加权与标准 GRPO 目标结合，得到以下损失函数：

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}, \quad (56)$$

其中，样本来自旧推理策略 π_0 。你可以验证，对这个目标求导，就能得到前面推导的 token 重加权策略梯度估计器，只是使用了 GRPO 优势与序列归一化。

现在可以给这一目标加入裁剪。令 $A^{(i,j)} = \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i}$ 表示提示 i 的第 j 个回答的优势，令 $w_t^{(i,j)} = \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}$ 表示第 t 个重要性重加权项。目标的裁剪版本如下：

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min\left(A^{(i,j)} w_t^{(i,j)}, A^{(i,j)} \text{clip}\left(w_t^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon]\right)\right), \quad (57)$$

其中, $\text{clip}(w, [1 - \varepsilon, 1 + \varepsilon]) = \min(\max(w, 1 - \varepsilon), 1 + \varepsilon)$ 将重要性权重裁剪到区间 $[1 - \varepsilon, 1 + \varepsilon]$ 。一种稍直观的目标写法如下 [J. Achiam, 2018]:

$$J_{\theta}^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \begin{cases} \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} & \text{若 } A^{(i,j)} \geq 0, \\ \max(w_t^{(i,j)}, 1 - \varepsilon) A^{(i,j)} & \text{若 } A^{(i,j)} < 0. \end{cases} \quad (58)$$

在这一目标下, 模型被鼓励提高优势动作的权重, 直到其权重达到旧策略的 $1 + \varepsilon$ 倍。此时, $\min$ 选择 $1 + \varepsilon$ 分支, 对它求导产生零梯度。负优势动作的情况类似: 模型被鼓励降低其权重, 直到达到旧策略的 $1 - \varepsilon$ 倍, 此时该动作的梯度变为零。因此, 求导后得到以下策略梯度估计器:

$$\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \text{mask}_t^{(i,j)} w_t^{(i,j)} A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) \quad (59)$$

其中掩码函数为:

$$\text{mask}_t^{(i,j)} = \begin{cases} \mathbb{I}\{w_t^{(i,j)} < 1 + \varepsilon\} & \text{若 } A^{(i,j)} \geq 0, \\ \mathbb{I}\{w_t^{(i,j)} > 1 - \varepsilon\} & \text{若 } A^{(i,j)} < 0. \end{cases} \quad (60)$$

它会屏蔽正优势下较大的重要性权重, 以及负优势下较小的重要性权重。

现在已经具备在损失函数中实现 `importance_reweighting_method = "noclip"` 和 `importance_reweighting_method = "grpo"` 所需的数学基础, 它们分别对应 $J_{\theta}^{\text{GRPO-off-policy-noclip}}$ 与 $J_{\theta}^{\text{GRPO-off-policy-clip}}$ 。与同策略情况一样, 损失实现应返回目标的负值, 使梯度下降执行我们想要的梯度上升更新。

题目 (`compute_policy_gradient_loss_off_policy`): 采用 token 级重加权的异策略策略梯度 (1 分)

提交内容: 更新方法 `compute_policy_gradient_loss`, 支持 `importance_reweighting_method = "noclip"` 或 `"grpo"`, 它们使用 `old_log_probs` 与 `cliprange` 参数。`old_log_probs` 表示生成 rollout 时所使用模型的逐 token 对数概率, 需要在训练脚本中增加几行代码来计算。`cliprange` 表示裁剪强度参数 ε 。

要测试代码, 请实现 `[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy`, 确保通过测试。

注意: 虽然 PPO 将裁剪解释为防止当前策略偏离旧策略过远, 也可以把它视为降低重要性重加权估计器方差的方法, 因为它压制了过大的重要性权重项。如果不太担心偏离旧策略、希望更激进地更新, 那么一种更简单直接的控方差方式, 是在梯度估计器中直接对重要性权重做上界裁剪:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}), \quad (61)$$

与 PPO / GRPO 不同，即使某个动作的权重被提高到超过 $1 + \varepsilon$ ，这里仍会对它产生非零梯度。CISPO 提出了这种更简单的裁剪过程 [MiniMax 等, 2025]。注意，它们还使用组内 token 数归一化，而不是逐序列长度；这里为了简化，写成了序列归一化目标。作为选做内容，你也可以尝试 CISPO，并与其他裁剪方法比较。

6.3 GSPO

前面已经看到，token 级重加权存在偏差，因为它忽略了前缀和后缀的重加权。在 GSPO 论文中 [C. Zheng 等, 2025]，作者发现 GRPO 不稳定，因而主张改用序列级重加权。为绕开序列级重要性权重是 L 项乘积这一问题，他们直接对该表达式取 $1/L$ 次幂。换言之，使用整条序列重要性权重的几何平均，而不是乘积进行重加权。得到的损失函数为：

$$J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \min(A^{(i,j)} s^{(i,j)}, A^{(i,j)} \text{clip}(s^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon])), \quad (62)$$

其中，序列级重要性权重 $s^{(i,j)}$ 在所有时间步之间共享：

$$s^{(i,j)} = \left(\prod_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_{\theta}(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} \right)^{\frac{1}{\text{len}(y^{(i,j)})}}. \quad (63)$$

注意，如果没有 $1/\text{len}(y^{(i,j)})$ 次幂，也没有裁剪，这个目标就成为序列级重要性重加权的策略梯度损失。取几何平均和裁剪都以引入偏差为代价降低方差。另一个值得注意的点是：由于几何平均中的指数，求梯度时会自然得到序列长度归一化。为简洁起见，下式忽略裁剪：

$$\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A^{(i,j)} \nabla_{\theta} s^{(i,j)} \quad (64)$$

$$= \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A^{(i,j)} s^{(i,j)} \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (65)$$

因此，若希望将 GSPO 用于你偏好的常数归一化 RL 算法（选做），就需要改变序列级重要性权重中的指数。

题目 (think_about_importance_reweighting)：思考重要性重加权 (2 分)

考虑三种可能用于异策略 RL 的重要性重加权策略：(a) 不做重要性重加权；(b) PPO / GRPO 风格、带裁剪的 token 级重要性重加权；(c) GSPO 风格、使用几何平均并带裁剪的序列级重要性重加权。从偏差—方差权衡的角度看，各方法处于怎样的位置？能否想到一些场景，使其中一种在直觉上优于另外两种？

提交内容：几句话的评论。

下面将实现 GSPO 损失。记得以数值稳定的方式计算几何平均。

题目 (`compute_policy_gradient_loss_off_policy_gspo`): 采用序列级重加权的异策略策略梯度 (1 分)

提交内容: 更新方法 `compute_policy_gradient_loss`, 支持 `importance_reweighting_method = "gspo"`, 它使用 `old_log_probs` 与 `cliprange` 参数。`old_log_probs` 表示生成 rollout 时所使用模型的逐 token 对数概率, 需要在训练脚本中增加几行代码计算。`cliprange` 表示裁剪强度参数 ϵ 。和前面一样, 应返回目标的负值, 使最小化损失对应梯度上升。

要测试代码, 请实现 `[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo`, 确保通过测试。

6.4 实验

现在已经具备更新 `grpo_train_step`、使其支持异策略训练的所有组件。

题目 (`grpo_train_step_off_policy`): 异策略 GRPO 训练步 (2.5 分)

提交内容: 更新 `grpo_train_step` 方法, 支持异策略参数 `importance_reweighting_method`、`old_log_probs`、`cliprange`。此时该函数应支持全部参数。

要测试代码, 请实现 `[adapters.run_grpo_train_step]`。然后运行 `uv run pytest -k test_grpo_train_step_off_policy`, 确认通过测试。

改成异策略训练是否真的更快? 稳定性会损失多少? 哪一种重加权/裁剪方法最好? 我们将在下面的实验中探索这些问题。相对于先前的 RL 运行, 请保持超参数不变, 唯一变化是现在采用 32 倍异策略设置, 即每个推理批次执行 32 次训练步:

```
rollout_batch_size = 256
train_batch_size = 8
gradient_accumulation_steps = 1
```

此外, 可以默认使用下面这些取自原始论文的裁剪范围, 即前面推导中的 ϵ 。作为选做, 也欢迎自行调节这些数值。

- `offpolicy_clip`: `cliprange = 0.2`。
- `offpolicy_gspo`: `cliprange = 3e-4`。

注意, 还必须在训练脚本中增加几行代码, 计算 `old_log_probs` 并传入重加权损失。

题目 (grpo_experiments_off_policy): 比较不同异策略算法的性能 (8 B200 小时; 10 分)

保持相对于标准 GRPO 训练运行的超参数不变 (学习率由你选择), 对以下每个变体使用 4 个随机种子运行实验。应使用零样本 `r1_zero` 提示。也可以不坚持标准版本, 而改用上一节中你最喜欢的 RL 算法变体。

- `offpolicy_naive`: 不再令推理和训练批次大小相等 (原本均为 256), 而是将训练批次大小减为推理批次大小的 $1/32$, 即 $256/32 = 8$ 。记得按相同比例减少梯度累积步数, 以继续充分利用 GPU。保持 `importance_reweighting_method = "none"`。
- `offpolicy_noclip`: 在 `offpolicy_naive` 基础上, 令 `importance_reweighting_method = "noclip"`。
- `offpolicy_clip`: 在 `offpolicy_naive` 基础上, 令 `importance_reweighting_method = "grpo"`。
- `offpolicy_gspo`: 在 `offpolicy_naive` 基础上, 令 `importance_reweighting_method = "gspo"`。

除已有指标外, 还要确保记录裁剪比例 (clip fraction)。

与此前完全同策略的 GRPO 运行相比, 这些方法表现如何? 改用异策略会如何影响训练稳定性及运行间方差? 两种裁剪方法的裁剪比例有什么差异, 哪一种看起来更稳定? 你认为哪些方法可能受益于更多超参数调优? 根据运行间方差, 你对结论有多大把握?

提交内容: 评论, 并以相应指标的图表支持论述。

7 尝试自己的策略梯度估计器

题目 (try_your_own): 尝试自己的策略梯度估计器 (10 分)

现在, 你已经见过文献中提出的多种策略梯度估计器, 也了解了它们背后的一些理论, 是时候提出自己的方法了! 下面是一些建议:

- 尝试不同的优势估计器, 从而对提示产生不同的重加权。
- 尝试不同的重要性重加权策略。
- 在优势估计器中尝试不同的奖励基线。

提出自己的策略梯度估计器, 然后在相同的 RL 设置, 即 OLMo-2-0425-1B 与 GSM8K 上运行, 并与已经尝试过的方法基线比较。使用多个随机种子, 固定问题参数, 使结果可比。相对于目前的方法, 请只改变一个因素: 也就是说, 本作业前面至少应有一种方法, 与你的方法仅有一处差别。

你提出的修改有什么理论依据或直觉? 它是否胜过基线?

提交内容: 评论, 并以相应指标的图表支持论述; 若数学推导有助于论证, 也可附上推导。

参考文献

- [1] J. Li 等,《DataComp-LM: 寻找下一代语言模型训练集》。[在线]。可获取于: <https://arxiv.org/abs/2406.11794>
- [2] T. OLMo 等,《2 OLMo 2 Furious》。[在线]。可获取于: <https://arxiv.org/abs/2501.00656>
- [3] K. Cobbe 等,《训练验证器以解决数学应用题》。[在线]。可获取于: <https://arxiv.org/abs/2110.14168>
- [4] DeepSeek-AI 等,《DeepSeek-R1: 通过强化学习激发大语言模型的推理能力》。[在线]。可获取于: <https://arxiv.org/abs/2501.12948>
- [5] W. Kwon 等,《利用 PagedAttention 为大语言模型服务实现高效内存管理》。2023。
- [6] Z. Liu 等,《理解 R1-Zero 式训练: 一种批判性视角》。[在线]。可获取于: <https://arxiv.org/abs/2503.20783>
- [7] OpenAI 等,《OpenAI o1 系统卡》。[在线]。可获取于: <https://arxiv.org/abs/2412.16720>
- [8] Z. Shao 等,《DeepSeekMath: 突破开放语言模型的数学推理极限》。[在线]。可获取于: <https://arxiv.org/abs/2402.03300>
- [9] J. Achiam,《深度强化学习入门 (Spinning Up in Deep Reinforcement Learning)》。2018。
- [10] N. Lambert,《从人类反馈中进行强化学习》。[在线]。可获取于: <https://rlhfbook.com/>
- [11] R. J. Williams,《用于联结主义强化学习的简单统计梯度跟随算法》, *Machine Learning*, 第 8 卷, 第 3-4 期, 第 229-256 页, 1992, doi: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696)。
- [12] F. Tajwar 等,《最大似然强化学习》。[在线]。可获取于: <https://arxiv.org/abs/2602.02710>
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford 和 O. Klimov,《近端策略优化算法》。[在线]。可获取于: <https://arxiv.org/abs/1707.06347>
- [14] T. Degris, M. White 和 R. S. Sutton,《异策略 Actor-Critic》。[在线]。可获取于: <https://arxiv.org/abs/1205.4839>
- [15] J. Achiam,《简化的 PPO-Clip 目标》。[在线]。可获取于: <https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWw20Ey/view>
- [16] MiniMax 等,《MiniMax-M1: 利用 Lightning Attention 高效扩展测试时计算》。[在线]。可获取于: <https://arxiv.org/abs/2506.13585>
- [17] C. Zheng 等,《组序列策略优化》。[在线]。可获取于: <https://arxiv.org/abs/2507.18071>